

```
# Iris Flower Classification - Task 1
```

```
print("Iris Flower Classification Project")  
print("Task 1 - Machine Learning")
```

```
Iris Flower Classification Project  
Task 1 - Machine Learning
```

```
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import LabelEncoder  
from sklearn.linear_model import LogisticRegression  
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.tree import DecisionTreeClassifier  
  
from sklearn.metrics import (  
    accuracy_score,  
    confusion_matrix,  
    classification_report  
)  
  
print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

Double-click (or enter) to edit

```
import pandas as pd  
  
df = pd.read_csv('/content/IRIS.csv')  
  
print("Dataset loaded successfully!")  
print("Dataset shape:", df.shape)  
  
df.head()
```

```
Dataset loaded successfully!  
Dataset shape: (150, 5)
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Next steps: [Generate code with df](#) [New interactive sheet](#)

Step 3: Dataset Information

```
print("Dataset Information:")
df.info()

print("\nDataset Shape:")
print(df.shape)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nColumn Names:")
print(df.columns.tolist())
```

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
Column Non-Null Count Dtype
--- -
0 sepal_length 150 non-null float64
1 sepal_width 150 non-null float64
2 petal_length 150 non-null float64
3 petal_width 150 non-null float64
4 species 150 non-null object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

Dataset Shape:
(150, 5)

Missing Values:
sepal_length 0
sepal_width 0
petal_length 0
petal_width 0
species 0
dtype: int64

Column Names:
['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']

Step 4: Statistical Analysis

```
df.describe()
```

	sepal_length	sepal_width	petal_length	petal_width	
count	150.000000	150.000000	150.000000	150.000000	
mean	5.843333	3.054000	3.758667	1.198667	
std	0.828066	0.433594	1.764420	0.763161	
min	4.300000	2.000000	1.000000	0.100000	
25%	5.100000	2.800000	1.600000	0.300000	
50%	5.800000	3.000000	4.350000	1.300000	
75%	6.400000	3.300000	5.100000	1.800000	
max	7.900000	4.400000	6.900000	2.500000	

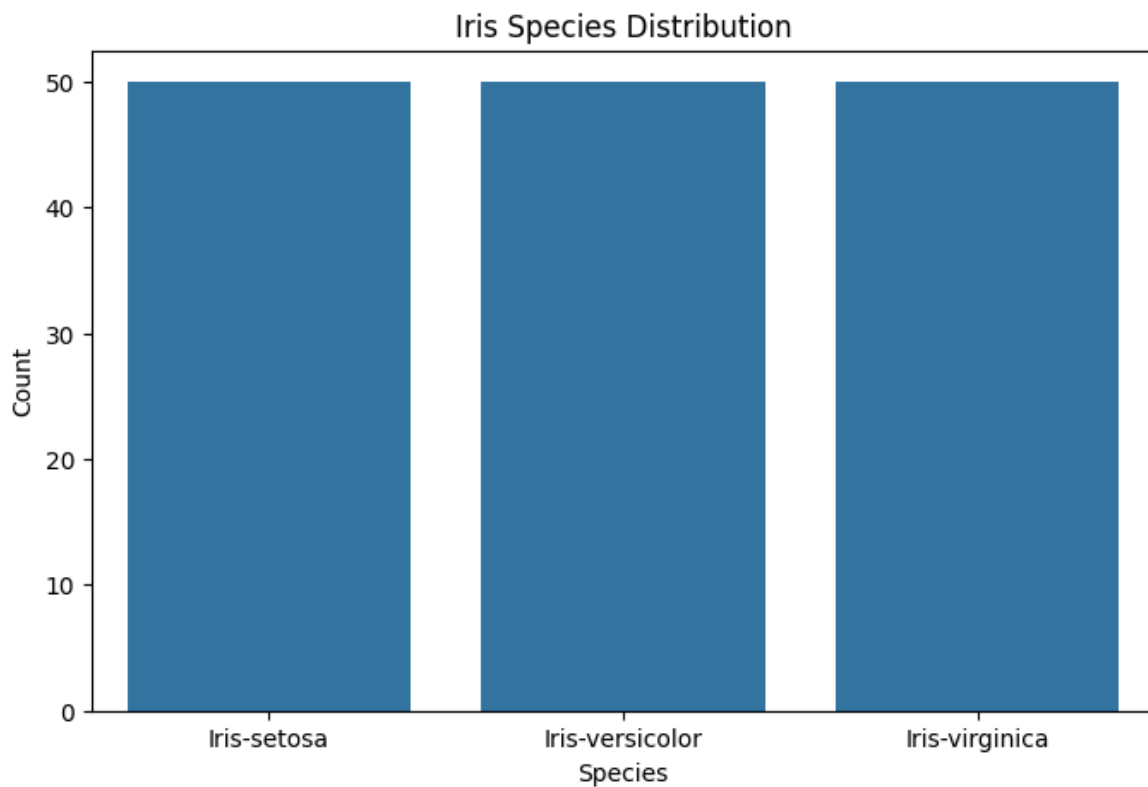
```
# Step 5: Species Distribution
```

```
print("Number of samples in each species:")  
print(df['species'].value_counts())
```

```
Number of samples in each species:  
species  
Iris-setosa      50  
Iris-versicolor  50  
Iris-virginica   50  
Name: count, dtype: int64
```

```
# Visualize species distribution
```

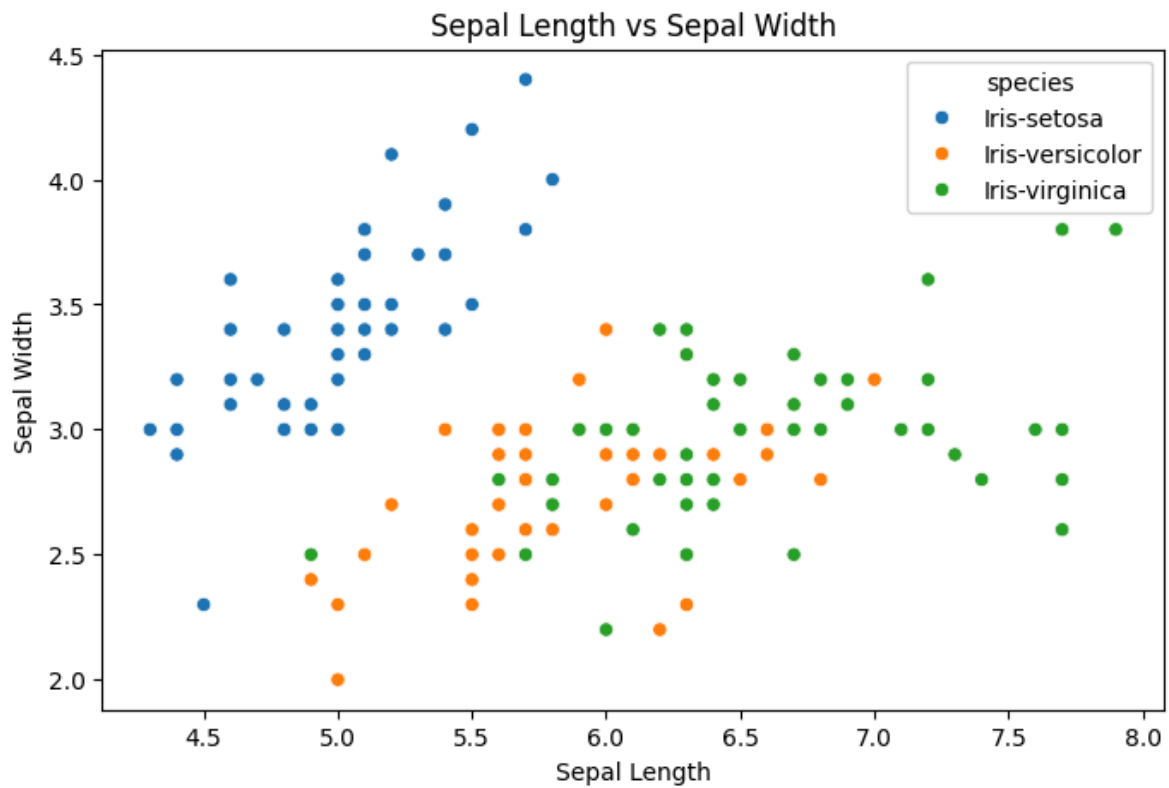
```
plt.figure(figsize=(8, 5))  
  
sns.countplot(data=df, x='species')  
  
plt.title('Iris Species Distribution')  
plt.xlabel('Species')  
plt.ylabel('Count')  
  
plt.show()
```



```
plt.figure(figsize=(8, 5))  
  
sns.scatterplot(  
    data=df,  
    x='sepal_length',  
    y='sepal_width',  
    hue='species'  
)  
plt.title('Sepal Length vs Sepal Width')
```

```
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
```

```
plt.show()
```

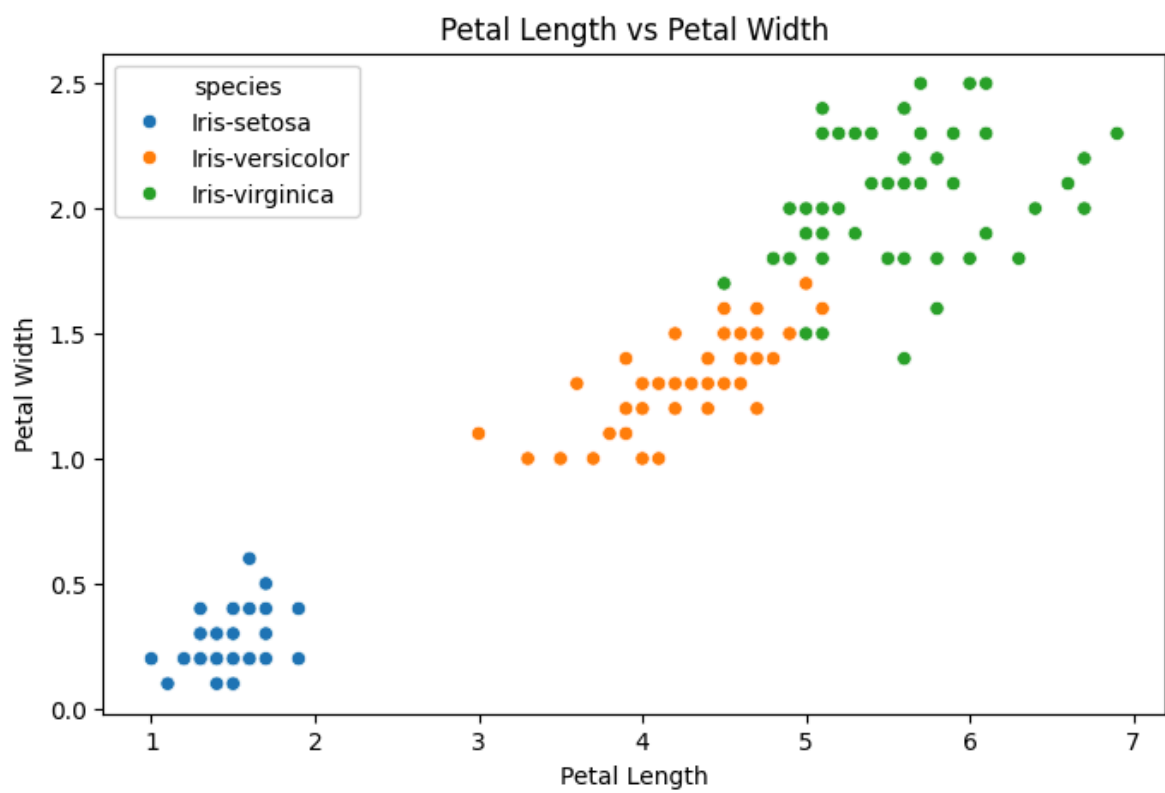


```
plt.figure(figsize=(8, 5))
```

```
sns.scatterplot(
    data=df,
    x='petal_length',
    y='petal_width',
    hue='species'
)
```

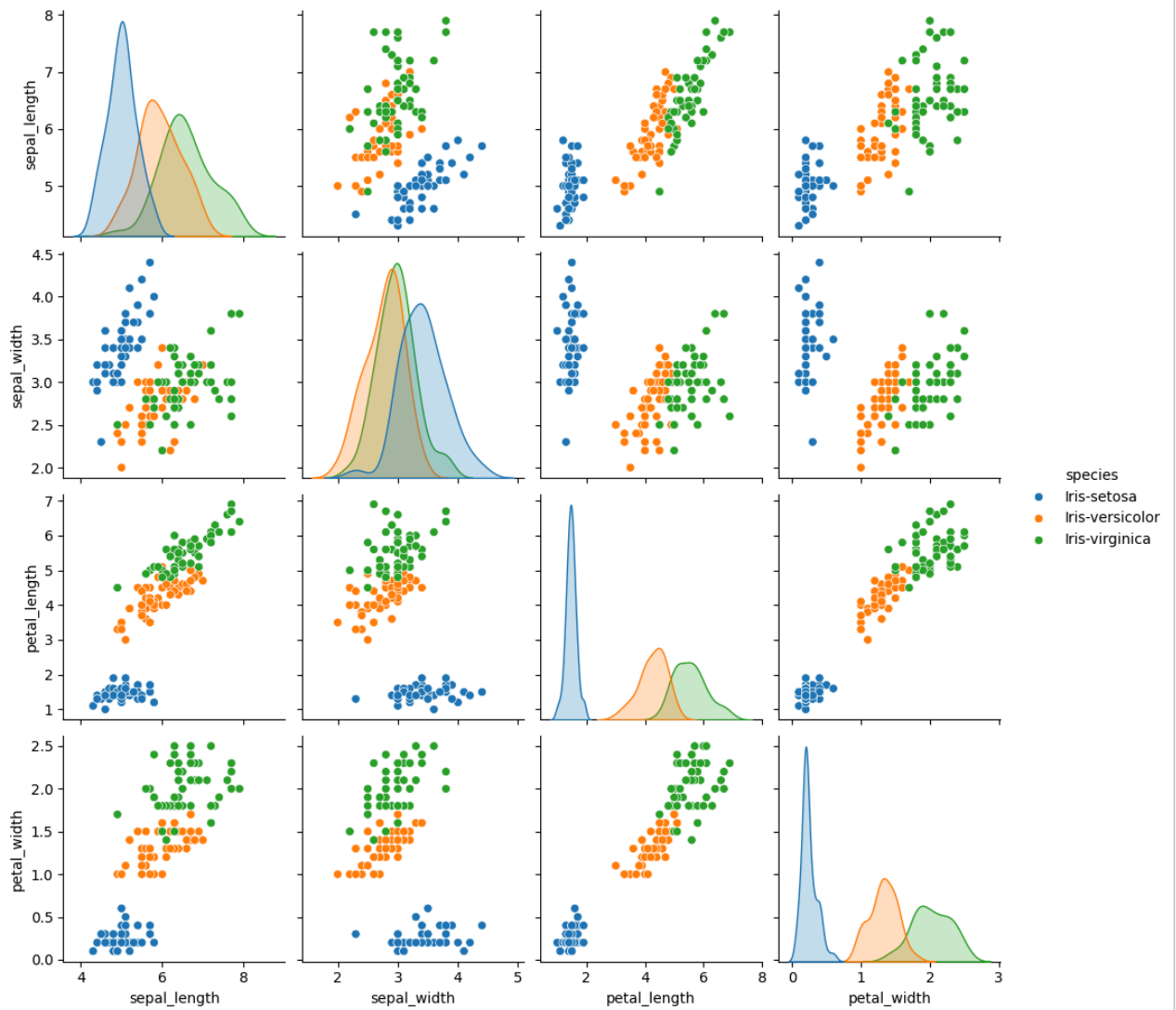
```
plt.title('Petal Length vs Petal Width')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')
```

```
plt.show()
```



Step 7A: Pair Plot

```
sns.pairplot(df, hue='species')  
plt.show()
```



Step 7B: Box Plots

```
plt.figure(figsize=(12, 6))
```

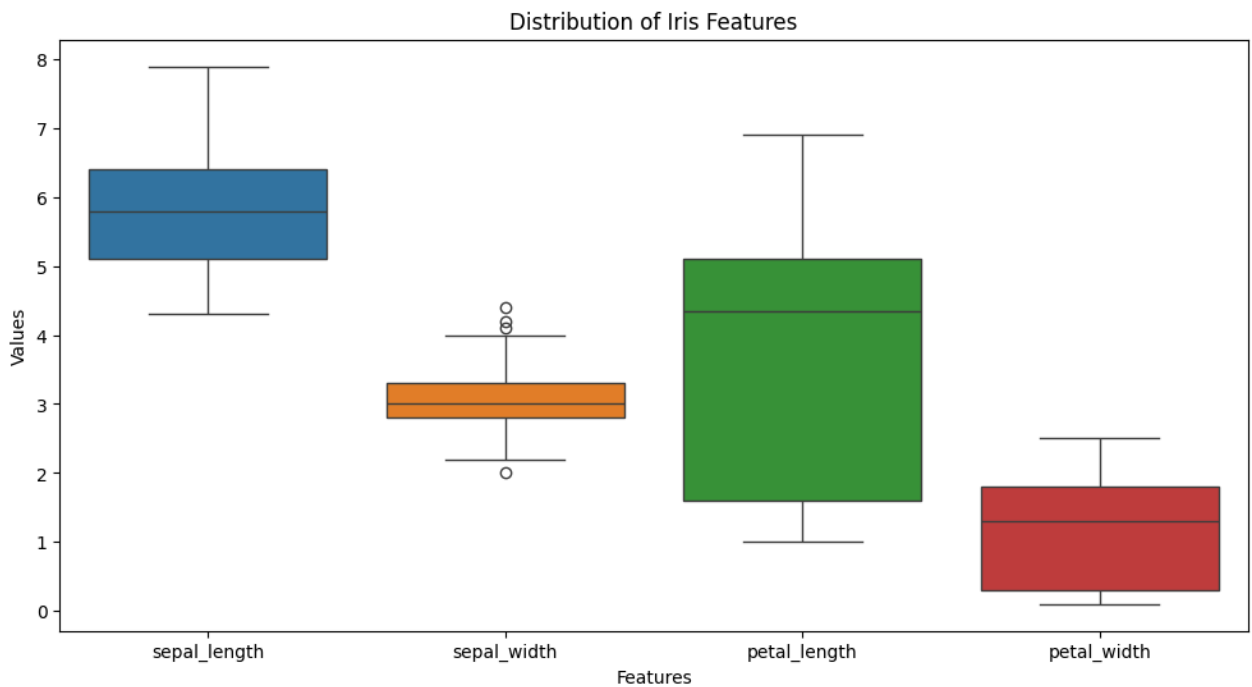
```
sns.boxplot(data=df)
```

```
plt.title('Distribution of Iris Features')
```

```
plt.xlabel('Features')
```

```
plt.ylabel('Values')
```

```
plt.show()
```



```
# Step 8: Encode the target variable

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df['species_encoded'] = le.fit_transform(df['species'])

print("Species mapping:")
for label, number in zip(le.classes_, le.transform(le.classes_)):
    print(label, "->", number)

df.head()
```

```
Species mapping:
Iris-setosa -> 0
Iris-versicolor -> 1
Iris-virginica -> 2
```

```
# Features (X) and Target (y)

X = df[['sepal_length',
        'sepal_width',
        'petal_length',
        'petal_width']]

y = df['species_encoded']

print("Features shape:", X.shape)
print("Target shape:", y.shape)
```

```
Features shape: (150, 4)
Target shape: (150,)
```

Step 9: Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (120, 4)
X_test shape: (30, 4)
y_train shape: (120,)
y_test shape: (30,)
```

Step 10: Train Machine Learning Models

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

# 1. k-NN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# 2. Logistic Regression
logistic = LogisticRegression(max_iter=200)
logistic.fit(X_train, y_train)

# 3. Decision Tree
decision_tree = DecisionTreeClassifier(random_state=42)
decision_tree.fit(X_train, y_train)

print("All 3 models trained successfully!")
```

```
All 3 models trained successfully!
```

Step 11: Make Predictions and Calculate Accuracy

```
from sklearn.metrics import accuracy_score

# Predictions
y_pred_knn = knn.predict(X_test)
y_pred_logistic = logistic.predict(X_test)
y_pred_tree = decision_tree.predict(X_test)

# Accuracy
accuracy_knn = accuracy_score(y_test, y_pred_knn)
accuracy_logistic = accuracy_score(y_test, y_pred_logistic)
accuracy_tree = accuracy_score(y_test, y_pred_tree)

print("Model Accuracy:")
```

```

print(f"k-NN: {accuracy_knn:.4f}")
print(f"Logistic Regression:{accuracy_logistic:.4f}")
print(f"Decision Tree: {accuracy_tree:.4f}")

```

```

Model Accuracy:
k-NN: 1.0000
Logistic Regression:0.9667
Decision Tree: 0.9333

```

Step 12: Classification Reports

```

from sklearn.metrics import classification_report

print("==== k-NN ====")
print(classification_report(
    y_test,
    y_pred_knn,
    target_names=le.classes_
))

print("==== Logistic Regression ====")
print(classification_report(
    y_test,
    y_pred_logistic,
    target_names=le.classes_
))

print("==== Decision Tree ====")
print(classification_report(
    y_test,
    y_pred_tree,
    target_names=le.classes_
))

```

```

==== k-NN ====
              precision    recall  f1-score   support

 Iris-setosa          1.00      1.00      1.00        10
 Iris-versicolor      1.00      1.00      1.00        10
 Iris-virginica        1.00      1.00      1.00        10

   accuracy                   1.00        30
  macro avg          1.00      1.00      1.00        30
 weighted avg          1.00      1.00      1.00        30

==== Logistic Regression ====
              precision    recall  f1-score   support

 Iris-setosa          1.00      1.00      1.00        10
 Iris-versicolor      1.00      0.90      0.95        10
 Iris-virginica        0.91      1.00      0.95        10

   accuracy                   0.97        30
  macro avg          0.97      0.97      0.97        30
 weighted avg          0.97      0.97      0.97        30

==== Decision Tree ====
              precision    recall  f1-score   support

 Iris-setosa          1.00      1.00      1.00        10
 Iris-versicolor      0.90      0.90      0.90        10
 Iris-virginica        0.90      0.90      0.90        10

```

accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

```
# Step 14: Final Model Comparison

results = pd.DataFrame({
    'Model': [
        'k-NN',
        'Logistic Regression',
        'Decision Tree'
    ],
    'Accuracy': [
        accuracy_knn,
        accuracy_logistic,
        accuracy_tree
    ],
    'Precision': [
        1.00,
        0.97
    ]
})

results
```

	Model	Accuracy	Precision	Recall	F1-Score	
0	k-NN	1.000000	1.00	1.00	1.00	
1	Logistic Regression	0.966667	0.97	0.97	0.97	
2	Decision Tree	0.933333	0.93	0.93	0.93	

Next steps: [Generate code with results](#) [New interactive sheet](#)

```
# Step 15: Save the best-performing model

import joblib

# Save k-NN model
joblib.dump(knn, 'iris_knn_model.joblib')

print("k-NN model saved successfully!")
```

k-NN model saved successfully!

```
import os

print(os.path.exists('iris_knn_model.joblib'))
```

True

```
# Step 16: Load the saved model

import joblib

loaded_model = joblib.load('iris_knn_model.joblib')

print("Model loaded successfully!")
```

Model loaded successfully!

```
# Example inference

new_flower = [[5.1, 3.5, 1.4, 0.2]]

prediction = loaded_model.predict(new_flower)

predicted_species = le.inverse_transform(prediction)

print("Predicted species:", predicted_species[0])
```

Predicted species: Iris-setosa
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X
warnings.warn(

✓ Iris Flower Classification

Dataset

The dataset contains 150 Iris flower samples and 5 columns.

Features

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

Target

- Species

Exploratory Data Analysis (EDA)

The following exploratory data analysis was performed:

- Dataset information was examined.
- Missing values were checked.

- Statistical summary was generated.
- Species distribution was visualized.
- Scatter plots were used to study class separability.
- Pair plot was used to visualize relationships between features.
- Box plots were used to examine feature distributions.

Machine Learning Algorithms

The following classification algorithms were trained and compared:

1. k-Nearest Neighbors (k-NN)
2. Logistic Regression
3. Decision Tree

Model Evaluation

The models were evaluated using the following metrics:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-Score

Results

The performance of the three classification models on the test set was:

Model	Accuracy
k-NN	100.00%
Logistic Regression	96.67%
Decision Tree	93.33%

The k-NN model achieved the highest accuracy on the selected test split.

Saved Model

The best-performing model on the selected test split was k-NN.

The trained model was saved using Joblib as:

```
iris_knn_model.joblib
```

Example Inference

The saved k-NN model can be loaded and used to predict the species of a new Iris flower.

```
import joblib

loaded_model = joblib.load('iris_knn_model.joblib')

new_flower = [[5.1, 3.5, 1.4, 0.2]]
```

```
prediction = loaded_model.predict(new_flower)
```

```
print("Predicted class:", prediction[0])
```

```
## Conclusion
```

Three machine learning classification algorithms were successfully trained and evaluated on

The models were evaluated using accuracy, confusion matrix, precision, recall, and F1-score

Among the tested models, k-NN achieved the highest accuracy on the selected test split with

The trained k-NN model was saved as a Joblib file and tested using an example inference.

```
# Step 18: Create requirements.txt
```

```
requirements = """numpy
pandas
matplotlib
seaborn
scikit-learn
joblib
"""
```

```
with open("requirements.txt", "w") as f:
    f.write(requirements)
```

```
print("requirements.txt created successfully!")
```

```
requirements.txt created successfully!
```

```
import os
```

```
print(os.path.exists("requirements.txt"))
```

```
True
```

✓ Executive Summary — Iris Flower Classification

Objective

The objective of this project is to build a machine learning classification model that predicts Iris flower species using sepal and petal measurements.

Dataset

The Iris dataset contains 150 samples and 5 columns. The four input features are sepal length, sepal width, petal length, and petal width. The target variable is the Iris species.

Methodology

The project followed these steps:

1. Loaded the Iris dataset.
2. Performed exploratory data analysis.
3. Checked missing values and statistical summaries.
4. Visualized species distribution and class separability.
5. Encoded the target species into numerical labels.
6. Split the dataset into 80% training data and 20% testing data.
7. Trained three classification algorithms.
8. Evaluated the models using accuracy, confusion matrix, precision, recall, and F1-score.
9. Saved the best-performing model using Joblib.
10. Tested the saved model using an example inference.

Models Used

Three machine learning classification algorithms were trained:

1. k-Nearest Neighbors (k-NN)
2. Logistic Regression
3. Decision Tree

Results

Model	Accuracy
k-NN	100.00%
Logistic Regression	96.67%
Decision Tree	93.33%

The k-NN model achieved the highest accuracy on the selected test split with an accuracy of 100%.

Conclusion

The Iris flower classification task was successfully completed. The three classification models were trained and evaluated using multiple performance metrics. The k-NN model achieved the highest accuracy on the selected test split and was saved as `iris_knn_model.joblib` for future inference.

Start coding or [generate](#) with AI.